

เอกสารประกอบการสอน สัปดาห์ที่ 2

From Problem Brief to a Shared Understanding of the Problem

ENGSE206 | Week 2: Stakeholder, System Context, Scope และ Ethics

หน่วยเรียน	หน่วยที่ 1: ฐานคิดของวิศวกรรมความต้องการและความเข้าใจปัญหา
บทเรียน	1.2 Problem Framing, Stakeholder Analysis, System Context, Scope และ Ethics
เวลาเรียน	ทฤษฎี 2 ชั่วโมง ปฏิบัติ/Workshop 3 ชั่วโมง
ผลลัพธ์สำคัญ	สร้างความเข้าใจร่วมต่อปัญหา ผู้เกี่ยวข้อง ขอบเขต และคำถามที่ต้องเก็บข้อมูลเพิ่มก่อนเริ่มเขียน SRS
ความเชื่อมโยง	รับต่อจาก Problem Brief v0.1 ใน Week 1 → ส่งต่อ Open Questions และ Stakeholder Targets ไปยัง Week 3



ENGSE206 Week 2: Stakeholder, Context และ Scope

จาก Problem Brief สู่กรอบปัญหาที่ชัดเจน

1

Problem Framing



- ปัญหาปัจจุบันคืออะไร
- ใครได้รับผลกระทบ
- ผลลัพธ์ที่อยากเห็นคืออะไร

2

Stakeholder Analysis



- ผู้ใช้หลัก
- เจ้าหน้าที่/ผู้ปฏิบัติงาน
- เจ้าของระบบ/ผู้อนุมัติ
- ผู้ได้รับผลกระทบทางอ้อม

3

System Context



- ระบบอยู่ตรงไหน
- ข้อมูลเข้า-ออกอะไร
- เกี่ยวข้องกับระบบ/หน่วยงานใด

4

Scope & Constraints



In Scope



Out of Scope



Constraints



Assumptions



Open Questions

5

Ethics / Privacy / AI



- เก็บข้อมูลเท่าที่จำเป็น
- ไม่ใส่ข้อมูลจริงที่อ่อนไหวลง repo
- ใช้ AI ช่วยได้แต่ต้องตรวจสอบ



หมายเหตุ

อย่าลืมพิจารณาผลกระทบต่อ
ผู้ใช้ สังคม และกฎหมายที่เกี่ยวข้อง
ตลอดการพัฒนา

Workshop Output



Stakeholder Map



Context Diagram



Scope Statement



GitHub Submission



เป้าหมายสัปดาห์นี้: รู้ว่าใครเกี่ยวข้อง ระบบอยู่ตรงไหน และยังไม่รู้อะไร

รูปที่ 2.1 ภาพรวมแนวคิด Stakeholder, Context, Scope และ Ethics ที่ต้องเข้าใจก่อนเริ่มเขียนเอกสาร SRS

ชื่อบทเรียน

- 1.2.1 การกำหนดกรอบปัญหา (Problem Framing): อะไรคือปัญหาที่แท้จริง ใครได้รับผลกระทบ และผลลัพธ์ใดที่ต้องการ
- 1.2.2 การวิเคราะห์ผู้มีส่วนได้ส่วนเสีย (Stakeholder Analysis): ผู้ใช้ เจ้าหน้าที่ ผู้ตัดสินใจ ผู้ดูแล และผู้ได้รับผลกระทบทางอ้อม
- 1.2.3 บริบทและขอบเขตของระบบ (System Context and Scope): ระบบอยู่ตรงไหน ทำอะไร และไม่ทำอะไร
- 1.2.4 ข้อจำกัด ข้อสมมติ คำถามเปิด และการคิดด้านจริยธรรม/ความเป็นส่วนตัวก่อนสร้างระบบ

วัตถุประสงค์การสอน

- อธิบายได้ว่าการทำความเข้าใจปัญหาก่อนออกแบบ solution ช่วยลดความเสี่ยงของการสร้างระบบผิดความต้องการอย่างไร
- วิเคราะห์และจำแนก stakeholder ของกรณีศึกษา พร้อมอธิบายเป้าหมาย ความต้องการ ข้อกังวล และอิทธิพลของแต่ละฝ่ายได้
- สร้าง System Context Diagram ระดับภาพรวมเพื่อแสดงระบบที่กำลังศึกษา ข้อมูลเข้า-ออก และหน่วยงาน/ระบบภายนอกที่เกี่ยวข้องได้
- กำหนด In Scope, Out of Scope, Constraints, Assumptions และ Open Questions ที่มีเหตุผลรองรับได้
- อธิบายได้ว่า artefact ของ Week 2 จะกลายเป็นส่วนตั้งต้นของ SRS และแผนเก็บความต้องการในสัปดาห์ถัดไปอย่างไร
- ตระหนักถึง privacy, ethics, security และการใช้ AI อย่างรับผิดชอบในงานวิศวกรรมซอฟต์แวร์

บทนำ: เหตุใด SRS จึงไม่ควรเริ่มด้วยการเขียนรายการฟังก์ชัน

เอกสาร Software Requirements Specification หรือ SRS มีหน้าที่ทำให้ผู้เกี่ยวข้องเข้าใจตรงกันว่า

“ระบบต้องทำอะไร ภายใต้เงื่อนไขใด และมีคุณภาพอย่างไร” อย่างไรก็ตาม การเริ่มเขียน SRS

ทันทีโดยที่ทีมยังไม่เข้าใจปัญหา ผู้เกี่ยวข้อง หรือขอบเขตของระบบ

มักทำให้เอกสารมีเพียงรายการฟังก์ชันที่เกิดจากการคาดเดา ไม่สามารถตรวจสอบได้ว่าใครต้องการอะไร

และเหตุใดสิ่งนั้นจึงจำเป็น

ก่อนเขียน SRS ทีมวิศวกรรมซอฟต์แวร์ต้องสร้าง shared understanding หรือความเข้าใจร่วมระหว่างสมาชิกทีม ผู้ใช้ เจ้าหน้าที่ และผู้ตัดสินใจเสียก่อน งานใน Week 2 จึงเป็นการสร้าง “รากฐานของ requirement” ผ่านการมองปัญหาอย่างเป็นระบบ ไม่ใช่การออกแบบหน้าจอหรือเลือกเทคโนโลยี

แนวคิดหลักของสัปดาห์นี้

SRS ที่ดีไม่ได้เกิดจากการเขียนเก่งเพียงอย่างเดียว แต่เกิดจากการเข้าใจปัญหา ผู้เกี่ยวข้อง และขอบเขตของระบบอย่างมีหลักฐานรองรับ

1. Requirement Engineering ก่อน SRS:

กระบวนการที่ช่วยป้องกันการสร้างระบบผิดเรื่อง

ทำไปเพื่ออะไร?

Requirement Engineering เป็นงานที่ช่วยเปลี่ยน “ความต้องการที่ยังคลุมเครือ” ให้กลายเป็นความเข้าใจร่วมและความต้องการที่ตรวจสอบได้ หากข้ามขั้นนี้ ทีมอาจสร้างสิ่งที่ทำงานได้ตามโปรแกรม แต่ไม่ช่วยแก้ปัญหาของผู้ใช้จริง

ผลลัพธ์ที่ได้คืออะไร?

ทีมได้เส้นทางเชิงเหตุผลจากปัญหา → ผู้เกี่ยวข้อง → ขอบเขต → คำถามที่ต้องค้นหา → หลักฐาน → requirement candidate

เชื่อมกับ SRS อย่างไร?

ส่วน Problem Statement, Stakeholder, Scope, Assumptions และ Glossary จะกลายเป็นบริบทสำคัญของ SRS และช่วยให้ข้อกำหนดรายข้อไม่หลุดจากปัญหาจริง

ในงานวิศวกรรมซอฟต์แวร์ เราไม่สามารถมอง requirement เป็นเพียง “รายการความสามารถของระบบ” ได้ เพราะ requirement มีทั้งเป้าหมายเชิงธุรกิจ กฎการทำงาน ข้อจำกัด คุณลักษณะด้านคุณภาพ และความคาดหวังที่อาจขัดแย้งกันของผู้เกี่ยวข้องหลายฝ่าย

สิ่งที่ทีมมักพูด	คำถามเชิงวิศวกรรมที่ต้องถามต่อ	เหตุผล
“ทำแอปจองห้อง”	ปัญหาการจองปัจจุบันคืออะไร? ใครได้รับผลกระทบ? ทำไมต้องรวมข้อมูล?	แยกปัญหาออกจาก solution ที่ยังพิสูจน์ไม่ได้
“อยากมีแรงเตือน”	ผู้ใช้พลาดข้อมูลเมื่อใด? ใครต้องได้รับแจ้ง? แจ้งผ่านช่องทางใด?	หาเหตุผลและเงื่อนไขก่อนเขียน requirement
“ระบบต้องใช้ง่าย”	ผู้ใช้กลุ่มใดทำงานได้บ่อยที่สุด? “ง่าย” วัดจากอะไร?	แปลงคำกว้างให้ตรวจสอบและออกแบบได้
“ต้องปลอดภัย”	ข้อมูลใดสำคัญ? ใครควรเข้าถึง? เกิดผลเสียอะไรหากข้อมูลรั่ว?	เชื่อม security กับความเสี่ยงจริงของระบบ

2. Problem Framing: ทำให้แน่ใจก่อนว่าเรากำลังแก้ปัญหาที่ถูกต้อง

ทำไปเพื่ออะไร?

Problem Framing ช่วยให้ทีมไม่รีบสร้าง solution จากความเชื่อหรือความชอบของตนเอง แต่เริ่มจากสถานการณ์ปัจจุบัน pain point ผลกระทบ และผลลัพธ์ที่ผู้เกี่ยวข้องต้องการจริง

ผลลัพธ์ที่ได้คืออะไร?

ได้ Problem Frame ที่อธิบาย Current Situation, Pain Points, Affected Stakeholders, Desired Outcome และ Initial Constraints

เชื่อมกับ SRS อย่างไร?

Problem Frame เป็นที่มาของส่วน Background, Business Need และ Problem Statement ใน SRS และช่วยตรวจว่า requirement ทุกข้อสนับสนุนเป้าหมายใด

หลักคิดสำคัญคือ “Problem ไม่เท่ากับ Solution” เช่น ประโยคว่า “ควรทำแอปจองพื้นที่” ไม่ได้อธิบายว่าอะไรคือปัญหา หากไม่รู้ปัญหา ทีมจะไม่สามารถตัดสินใจว่าแอป เว็บไซต์ แบบฟอร์ม หรือการปรับขั้นตอนทำงานแบบอื่นจะเหมาะสมที่สุด

จากคำพูดทั่วไป	วิเคราะห์ให้เป็น Problem Frame	สิ่งที่ได้เพิ่ม
“จองพื้นที่ยาก”	ผู้ใช้ต้องสอบถามหลายช่องทาง ข้อมูลห้องว่างไม่เป็นปัจจุบัน และเกิดการจองซ้ำ	เห็น pain point ที่ต้องแก้: แหล่งข้อมูลกระจายและสถานะไม่โปร่งใส
“แจ้งซ่อมช้า”	ผู้พบปัญหาไม่รู้ว่าติดต่อใคร เจ้าหน้าที่ตรวจสอบย้อนหลังยาก และผู้แจ้งไม่ทราบสถานะ	เห็นทั้งปัญหาของผู้แจ้งและปัญหาการติดตามงานของเจ้าหน้าที่
“นัดอาจารย์ลำบาก”	ตารางว่างไม่ชัด มีการทักหลายช่องทาง และการยกเลิกนัดไม่ถูกสื่อสารอย่างทั่วถึง	เห็น need เรื่องข้อมูลเวลา การยืนยันนัด และการสื่อสารการเปลี่ยนแปลง

ตัวอย่างสถานการณ์ใกล้ตัว: ระบบจองพื้นที่ทำงานกลุ่ม

นักศึกษาต้องถามเจ้าหน้าที่ผ่าน LINE ดูป้ายหน้าห้อง และสอบถามอุปกรณ์แยกกัน เมื่อมีการเปลี่ยนแปลง คนที่สนใจห้องเดียวกันอาจไม่ทราบ ทำให้เกิดการจองชนกันและความขัดแย้ง ปัญหาหลักคือ “ข้อมูลไม่รวมศูนย์และกระบวนการไม่โปร่งใส” ไม่ใช่เพียง “ยังไม่มีแอป”

คำถามช่วยทำ Problem Frame

- ปัจจุบันผู้ใช้ทำงานนี้อย่างไร และขั้นตอนใดทำให้เสียเวลา สับสน หรือผิดพลาด?
- ใครได้รับผลกระทบโดยตรง และใครได้รับผลกระทบทางอ้อม?
- หากไม่ปรับปรุง จะเกิดผลเสียหรือความเสี่ยงใดต่อผู้ใช้ หน่วยงาน หรือข้อมูล?
- ผลลัพธ์ที่ต้องการคืออะไร โดยยังไม่รับกำหนดวิธีแก้เชิงเทคนิค?
- มีข้อจำกัดใดที่ต้องยอมรับตั้งแต่แรก เช่น เวลา งบประมาณ บุคลากร หรือความพร้อมของข้อมูล?

3. Stakeholder Analysis: เพราะผู้ใช้ไม่ใช่ผู้เกี่ยวข้องทั้งหมด

ทำไปเพื่ออะไร?

ระบบหนึ่งระบบมีหลายฝ่ายที่มีเป้าหมายต่างกัน ผู้ใช้ต้องการความสะดวก เจ้าหน้าที่ต้องการลดงานซ้ำ
เจ้าของระบบต้องการความโปร่งใส และผู้ดูแลระบบต้องการความปลอดภัย หากทีมฟังเพียงฝ่ายเดียว requirement
มักไม่สมดุล

ผลลัพธ์ที่ได้คืออะไร?

ได้ stakeholder inventory, stakeholder profile และการจัดลำดับความสำคัญของ stakeholder ตาม interest และ
influence

เชื่อมกับ SRS อย่างไร?

ข้อมูลนี้จะช่วยกำหนดผู้ให้ข้อมูลในการเก็บ requirement และช่วยให้ SRS ระบุบทบาทผู้ใช้ สิทธิ กฎ
และข้อจำกัดได้ครบถ้วน

Stakeholder คือบุคคล กลุ่ม หน่วยงาน หรือระบบภายนอกที่มีผลต่อระบบ ได้รับผลกระทบจากระบบ
หรือมีอำนาจตัดสินใจเกี่ยวกับระบบ บาง stakeholder ไม่ได้กดใช้หน้าจอเลย แต่มีอิทธิพลสูงต่อกติกา งบประมาณ
หรือความปลอดภัยของข้อมูล

กลุ่ม stakeholder	ตัวอย่างในระบบจองพื้นที่	สิ่งที่ทีมควรเรียนรู้จากกลุ่มนี้
Primary User	นักศึกษาที่ต้องการจองพื้นที่	กระบวนการจองจริง ปัญหาที่เจอ ความเร่งด่วน และสิ่งที่ทำให้ใช้งานไม่สำเร็จ
Operational Staff	เจ้าหน้าที่ห้อง/ผู้ดูแลพื้นที่	กฎการอนุมัติ ขั้นตอนยืนยัน การจัดการเหตุขัดข้อง และข้อมูลที่ต้องบันทึก
System Owner / Decision Maker	หัวหน้าหน่วยงานหรือผู้รับผิดชอบพื้นที่	นโยบาย เป้าหมาย ภาพรวมความสำเร็จ ข้อจำกัด และรายงานที่ต้องการ
Administrator / IT	ผู้ดูแลบัญชีหรือระบบ	สิทธิ์การเข้าถึง การดูแลข้อมูล การเชื่อมต่อ และข้อควรระวังด้านความปลอดภัย
Indirect Stakeholder	นักศึกษากลุ่มอื่น ผู้ใช้งานพื้นที่ร่วมกัน	ผลกระทบจากกฎการจอง ความเป็นธรรม และข้อมูลที่ควรเห็นหรือไม่ควรเห็น

Interest–Influence Matrix

การจัด stakeholder ตามระดับ “ความสนใจ” และ “อิทธิพลต่อการตัดสินใจ”

ช่วยให้ทีมเลือกวิธีสื่อสารและลำดับความสำคัญได้เหมาะสม ตัวอย่างเช่น

เจ้าหน้าที่ห้องอาจสนใจสูงและรู้รายละเอียดการทำงานมาก

ส่วนผู้บริหารอาจใช้งานน้อยแต่มีอิทธิพลสูงต่อกฎและทรัพยากร

กลุ่มใน Matrix	วิธีปฏิบัติของทีม	เหตุผล
Interest สูง / Influence สูง	เก็บข้อมูลเชิงลึกและนำเสนอความคืบหน้าเป็นระยะ	เป็นผู้ให้ข้อมูลสำคัญและมีอำนาจต่อทิศทางระบบ
Interest สูง / Influence ต่ำ	เก็บ pain point และรายละเอียดการใช้งานจริง	สะท้อนปัญหาหน้าที่ requirement ต้องตอบ
Interest ต่ำ / Influence สูง	ขอความชัดเจนเรื่องนโยบาย ข้อจำกัด และเกณฑ์สำเร็จ	ช่วยลดความเสี่ยงที่ระบบขัดกับกฎหรือการตัดสินใจ
Interest ต่ำ / Influence ต่ำ	ติดตามผลกระทบและความต้องการเฉพาะกรณี	ไม่ควรละเลยหากระบบส่งผลต่อความเป็นธรรมหรือความปลอดภัย

ความเข้าใจผิดที่พบบ่อย

การใส่เพียง “นักศึกษา” และ “อาจารย์” ใน stakeholder map ยังไม่เพียงพอ ต้องระบุบทบาทที่แตกต่างกัน เช่น ผู้จอง
ผู้อนุมัติ ผู้ดูแลพื้นที่ ผู้จัดการข้อมูล และผู้ได้รับผลกระทบ เพราะแต่ละบทบาทมีเป้าหมายและข้อมูลที่ต่างกัน

4. System Context: มองระบบในภาพรวมก่อนลงรายละเอียด

ทำไปเพื่ออะไร?

System Context Diagram ช่วยให้ทีมเห็น system boundary หรือเส้นแบ่งระหว่าง “สิ่งที่ระบบของเรารับผิดชอบ” กับ
“สิ่งที่อยู่นอกระบบแต่เกี่ยวข้อง” ลดความสับสนเมื่อเริ่มออกแบบ use case หรือ requirement

ผลลัพธ์ที่ได้คืออะไร?

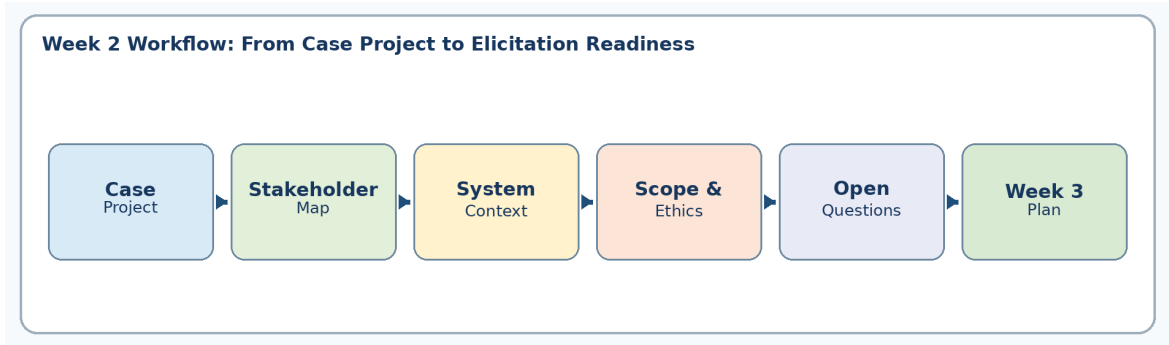
ได้ Context Diagram ที่แสดง System of Interest, External Entities และข้อมูลที่แลกเปลี่ยนกันในระดับภาพรวม

เชื่อมกับ SRS อย่างไร?

Context Diagram จะช่วยกำหนด actors, external interfaces, data flows และส่วน Integration/Assumptions ใน SRS ในอนาคต

ในระดับนี้ยังไม่ต้องระบุฐานข้อมูล ตาราง หรือหน้าจอ เป้าหมายคือการมอง “ระบบในโลกจริง” ว่าอยู่ท่ามกลางใครและแลกเปลี่ยนข้อมูลอะไรบ้าง ยิ่ง boundary ชัด ทีมยังสามารถกำหนด scope และวางแผนเก็บ requirement ได้แม่นยำ

องค์ประกอบ	ควรถามอะไร	ตัวอย่างระบบจองพื้นที่
System of Interest	สิ่งใดคือระบบที่ทีมกำลังศึกษา?	ระบบจองพื้นที่ทำงานกลุ่มและอุปกรณ์
External Entity	ใครหรือระบบใดอยู่นอก boundary แต่ต้องติดต่อ?	นักศึกษา เจ้าหน้าที่ห้อง หน่วย IT ระบบบัญชีมหาวิทยาลัย
Data / Information Flow	ข้อมูลใดเข้า-ออกระหว่างระบบและ entity?	คำขออนุมัติ สถานะการอนุมัติ ตารางว่าง การจองเดือน รายงานการใช้พื้นที่
Boundary	สิ่งใดที่ระบบนี้ไม่ต้องรับผิดชอบ?	การจัดซื้ออุปกรณ์ การซ่อมอาคาร ระบบบัญชีการเงินเต็มรูปแบบ



รูปที่ 2.2 ตัวอย่างกระบวนการจาก Problem Brief ไปสู่ artefact ที่ใช้พื้นฐานของ SRS

5. Scope, Constraints, Assumptions และ Open Questions: เครื่องมือควบคุมความเสี่ยงก่อนเริ่มเขียน SRS

ทำไปเพื่ออะไร?

โครงการซอฟต์แวร์มักล้มเหลวเมื่อขอบเขตขยายไม่หยุดหรือทีมคิดว่า “ทุกอย่างต้องทำได้หมด” การกำหนด scope และสิ่งที่ยังไม่รู้ช่วยให้ทีมทำงานได้พอดีกับเวลาและมีแผนลดความเสี่ยง

ผลลัพธ์ที่ได้คืออะไร?

ได้ Scope Statement ที่มี In Scope, Out of Scope, Constraints, Assumptions และ Open Questions พร้อมเหตุผล

เชื่อมกับ SRS อย่างไร?

ข้อมูลนี้ใช้เป็นขอบเขตของ SRS, acceptance criteria ระดับสูง, risk log และแผน requirement elicitation ใน Week 3

แนวคิด	คำถามที่ช่วยเขียน	ตัวอย่าง
In Scope	ระบบเวอร์ชันนี้ต้องช่วยให้ใครทำอะไรได้?	ดูสถานะพื้นที่, ส่งคำขอจอง, ยกเลิก, รับสถานะคำขอ
Out of Scope	เรื่องใดสำคัญแต่ยังไม่ทำในโครงการนี้?	ชำระเงิน, จัดซื้ออุปกรณ์, เชื่อม ERP, วิเคราะห์พยากรณ์ขั้นสูง
Constraints	มีข้อจำกัดจากเวลา นโยบาย เทคโนโลยี หรือทรัพยากรใด?	ดำเนินงานใน 1 เทอม, ใช้ข้อมูลจำลอง, ไม่เชื่อมข้อมูลส่วนบุคคลจริง
Assumptions	ทีมเชื่อว่าอะไรเป็นจริง แต่ยังไม่หลักฐาน?	ผู้ใช้ทุกคนมีบัญชีมหาวิทยาลัย, ห้องมีรหัสชัดเจน
Open Questions	เรื่องใดหากตอบผิดแล้วจะทำให้ requirement ผิด?	การจองต้องอนุมัติหรือไม่? จองล่วงหน้าได้กี่วัน? ใครยกเลิกแทนได้?

Open Questions มีความสำคัญเป็นพิเศษ เพราะเป็นสะพานระหว่าง Week 2 และ Week 3

ทีมจะไม่ปล่อยให้คำถามค้างอยู่ในหัว

แต่เปลี่ยนคำถามเหล่านั้นให้กลายเป็นแผนเก็บหลักฐานที่มีผู้รับผิดชอบและวิธีการชัดเจน

6. Ethics, Privacy, Security และ Responsible AI:

คุณภาพของระบบไม่ได้วัดจากฟังก์ชันเพียงอย่างเดียว

ทำไปเพื่ออะไร?

งานซอฟต์แวร์ส่งผลต่อผู้คน ข้อมูล และความเป็นธรรม การคิดเรื่อง privacy และ ethics

ตั้งแต่ต้นช่วยป้องกันไม่ให้ทีมออกแบบระบบที่ทำให้ผู้ใช้เสียเปรียบหรือเสี่ยงต่อการเปิดเผยข้อมูลโดยไม่จำเป็น

ผลลัพธ์ที่ได้คืออะไร?

ได้ Privacy/Ethics Note ที่ระบุข้อมูลที่จำเป็น ความเสี่ยงเบื้องต้น ผู้มีสิทธิ์เข้าถึง และหลักการใช้ AI ในงานของทีม

เชื่อมกับ SRS อย่างไร?

ประเด็นเหล่านี้จะต่อยอดเป็น non-functional requirements เช่น confidentiality, access control, auditability และ usability ใน SRS

- Data minimization: เก็บเฉพาะข้อมูลที่จำเป็นต่อวัตถุประสงค์ของระบบ
- Role-based access: ผู้ใช้แต่ละบทบาทควรเห็นและแก้ไขข้อมูลได้เท่าที่จำเป็น
- Transparency: ผู้ใช้ควรเข้าใจว่าสถานะใดเกิดจากใคร และข้อมูลใดถูกใช้เพื่ออะไร
- Fairness: กฎของระบบไม่ควรเอาเปรียบผู้ใช้บางกลุ่มโดยไม่มีเหตุผล
- Responsible AI: AI ช่วยร่าง สรุปรูป หรือจัดโครงสร้างได้ แต่ต้องไม่แต่งข้อเท็จจริงแทน stakeholder และต้องตรวจทานผลเสมอ

ตัวอย่างการคิดเชิงจริยธรรม

ระบบนัดอาจารย์ที่ปรึกษาอาจต้องบันทึกเหตุผลการขอพบ แต่บางเหตุผลเกี่ยวข้องกับสุขภาพหรือเรื่องส่วนตัว ดังนั้นทีมควรถามว่า “จำเป็นต้องเก็บรายละเอียดระดับใด?” “ใครควรเข้าถึงได้?” และ “หากไม่เก็บรายละเอียด ระบบยังช่วยจัดคิวได้หรือไม่?”

7. เส้นทางจาก Week 2 ไปสู่ SRS

Artefact ที่ได้ใน Week 2	ความหมายเชิงวิศวกรรม	ส่วนที่ใช้ต่อใน SRS/งานถัดไป
Problem Frame	อธิบายปัญหา เป้าหมาย และผลกระทบ	Background, Business Need, System Objective
Stakeholder Map / Profile	ระบุบทบาท เป้าหมาย และข้อกังวลของฝ่ายต่าง ๆ	User Classes, User Characteristics, Stakeholder Needs
System Context Diagram	กำหนด boundary และความสัมพันธ์ภายนอก	System Context, Interfaces, Actors
Scope Statement	ควบคุมสิ่งที่จะทำและไม่ทำ	Scope, Out of Scope, Release Boundary
Constraints / Assumptions	เปิดเผยเงื่อนไขที่มีผลต่อการออกแบบ	Constraints, Assumptions, Risks
Open Questions	ระบุสิ่งที่ต้องเก็บ evidence เพิ่ม	Elicitation Plan, Interview Guide, Issue Log

สรุปสำหรับนักศึกษา

Week 2 ไม่ใช่งานเอกสารเพื่อให้ครบไฟล์ แต่เป็นการสร้างเหตุผลและหลักฐานตั้งต้นให้ requirement ใน SRS ทุกข้อสามารถตอบได้ว่า “เกิดจากปัญหาใด ใครต้องการ และทีมรู้ได้อย่างไร”

8. Workshop และการส่งงานผ่าน GitHub

Workshop ของสัปดาห์นี้ใช้ Case Project ของแต่ละกลุ่มเป็นพื้นที่ฝึกปฏิบัติ นักศึกษาทำงานกลุ่ม 3-4 คน ใช้ repository เป็นพื้นที่บันทึก artefact เวอร์ชันต่าง ๆ และนำเสนอแบบ Tabletop Studio เพื่อให้เกิดการสื่อสารและ feedback ระหว่างทาง

ช่วงกิจกรรม	สิ่งที่ทำ	หลักฐานใน GitHub
เตรียมโจทย์	ทบทวน Problem Brief และ Case Card	อัปเดตข้อสังเกตใน docs/02-stakeholder-context-scope.md
วิเคราะห์ stakeholder	ระบุ stakeholder, interest, influence และ concern	Stakeholder inventory/profile ในเอกสาร
วาด context	กำหนด system boundary และ data flow ระดับสูง	diagrams/context/system-

ช่วงกิจกรรม	สิ่งที่ทำ	หลักฐานใน GitHub
		context.drawio และ .png
กำหนด scope	เขียน in/out scope, assumptions, constraints, open questions	Scope statement ในเอกสาร
Tabletop Studio	นำเสนอ 5 นาที รับ feedback และแก้ไข	feedback/week-02-peer-feedback.md
ส่งงาน	อัปเดต worklog, submission file, commit และ push	submissions/week-02-submission.md

แนวคำถามสำหรับการนำเสนอ 5 นาที

- ปัญหาปัจจุบันและผลกระทบที่กลุ่มเข้าใจคืออะไร?
- stakeholder ที่สำคัญที่สุด และเหตุใด?
- ขอบเขตของระบบอยู่ตรงไหน? เรื่องใดเป็น out of scope?
- open question ที่เสี่ยงที่สุดของกลุ่มคืออะไร และจะนำไปทำอะไรต่อใน Week 3?

9. Self-check ก่อนจบสัปดาห์

- ฉันสามารถอธิบายความแตกต่างระหว่าง problem และ solution ได้หรือไม่?
- ทีมระบุ stakeholder ได้มากกว่าผู้ใช้หลักเพียงกลุ่มเดียวหรือไม่?
- Context Diagram ของทีมแสดง boundary และข้อมูลเข้า-ออกชัดเจนหรือไม่?
- In Scope และ Out of Scope ของทีมมีเหตุผล ไม่กว้างเกินเวลาเรียนหรือไม่?
- Open Questions ของทีมสามารถนำไปสร้าง Elicitation Plan ใน Week 3 ได้หรือไม่?
- ทีมหลีกเลี่ยงการบันทึกข้อมูลส่วนบุคคลจริงและตรวจการใช้ AI อย่างรับผิดชอบหรือไม่?

บรรณานุกรมสำหรับอ่านเพิ่มเติม

- Sommerville, I. (2016). Software Engineering (10th ed.). Pearson.
- Wiegers, K. E., & Beatty, J. (2013). Software Requirements (3rd ed.). Microsoft Press.

- Robertson, S., & Robertson, J. (2012). Mastering the Requirements Process (3rd ed.). Addison-Wesley.
- ISO/IEC/IEEE 29148:2018. Systems and software engineering - Life cycle processes - Requirements engineering.